

Linux for DevOps — Detailed Notes

A practical, production-minded guide to the Linux skills every DevOps engineer uses daily: navigation, files, permissions, processes, services, networking, firewalls, SSH, logs, scripting, troubleshooting, and security. Expanded from the *Linux Quickstart Cheat Sheet for DevOps*.

Table of Contents

1. [Why Linux for DevOps](#)
2. [Filesystem Hierarchy](#)
3. [Navigation](#)
4. [File & Directory Management](#)
5. [Viewing, Searching & Text Processing](#)
6. [Pipes, Redirection & Shell Basics](#)
7. [Users, Groups & sudo](#)
8. [Permissions & Ownership](#)
9. [Package Management](#)
10. [Processes & Job Control](#)
11. [systemd: Services, Timers & Logs](#)
12. [System Monitoring & Performance](#)
13. [Disk, Storage & Filesystems](#)
14. [Networking](#)
15. [Firewall \(UFW, firewalld, iptables\)](#)
16. [SSH & Remote Access](#)
17. [cURL & API Testing](#)
18. [Archiving & Compression](#)
19. [Scheduling: cron & systemd timers](#)
20. [Environment Variables & Config Files](#)
21. [Bash Scripting for Automation](#)
22. [Logging & Log Management](#)
23. [Security Hardening Checklist](#)
24. [Linux Internals Behind Containers & Kubernetes](#)
25. [Troubleshooting Playbook](#)
26. [Time, Date & NTP](#)
27. [TLS/SSL & Certificates](#)
28. [Nginx & Reverse Proxy Essentials](#)
29. [Git for DevOps](#)

- 30. [Docker on Linux](#)
 - 31. [Kubernetes Quick Ops \(kubectl\)](#)
 - 32. [Backup & Recovery](#)
 - 33. [Advanced Debugging Tools](#)
 - 34. [SELinux & AppArmor](#)
 - 35. [Boot Process, Targets & Recovery](#)
 - 36. [Infrastructure as Code & Configuration Management](#)
 - 37. [CI/CD on Linux](#)
 - 38. [Master Cheat Sheet](#)
 - 39. [Best Practices & Golden Rules](#)
 - 40. [Practice Labs](#)
 - 41. [Interview Questions & Answers](#)
 - 42. [Glossary](#)
-

1. Why Linux for DevOps

- Most servers, cloud VMs, containers, and Kubernetes nodes run Linux.
- CI/CD runners, build agents, and automation tools are overwhelmingly Linux-based.
- Almost everything is **text-based and scriptable**: configs, logs, and system state are files.
- Remote work is done through **SSH + terminal**, so command-line fluency is essential.

Common distributions

Family	Distros	Package manager	Typical use
Debian	Ubuntu, Debian	<code>apt</code>	Cloud VMs, dev servers, containers
Red Hat	RHEL, Rocky, AlmaLinux, Fedora, CentOS Stream	<code>dnf</code> / <code>yum</code>	Enterprise servers
SUSE	SLES, openSUSE	<code>zypper</code>	Enterprise
Alpine	Alpine	<code>apk</code>	Minimal container images
Arch	Arch	<code>pacman</code>	Power users

Check which one you are on:

```
bash
```

```
cat /etc/os-release
uname -a           # kernel + architecture
hostnamectl        # hostname, OS, kernel, virtualization
```

2. Filesystem Hierarchy

Everything lives under the root directory (/). Linux follows the Filesystem Hierarchy Standard (FHS).

Path	Purpose
/	Root of the entire filesystem
/home	User home directories (/home/ubuntu)
/root	Home directory of the root user
/etc	System-wide configuration (nginx, ssh, fstab, hosts)
/var	Variable data: logs (/var/log), caches, spools, databases
/var/log	System and application logs
/opt	Optional / third-party software (/opt/myapp)
/usr	User programs and libraries (/usr/bin, /usr/local)
/bin, /sbin	Essential binaries (often symlinks to /usr/bin, /usr/sbin)
/tmp	Temporary files (cleared on reboot, world-writable)
/dev	Device files (/dev/sda, /dev/null)
/proc	Virtual FS exposing process and kernel info
/sys	Virtual FS exposing kernel/device settings
/mnt, /media	Mount points for extra disks / removable media
/srv	Data served by the system (web, FTP)
/boot	Kernel and bootloader files
/run	Runtime state (PID files, sockets) — tmpfs

Key idea: "Everything is a file" — devices, processes, and sockets are exposed as files.

```
bash
```

```
cat /proc/cpuinfo      # CPU details
cat /proc/meminfo      # memory details
cat /proc/loadavg      # load averages
ls /proc/$$            # info about the current shell process
```

3. Navigation

Command	What it does	Example
<code>pwd</code>	Print working directory	<code>pwd</code> → <code>/home/ubuntu</code>
<code>ls</code>	List directory contents	<code>ls /etc</code>
<code>ls -lah</code>	Long format, all files (incl. hidden), human-readable sizes	<code>ls -lah /var/log</code>
<code>cd <path></code>	Change directory	<code>cd /etc/nginx</code>
<code>cd ..</code>	Up one level	<code>cd ..</code>
<code>cd ~</code>	Go to home directory	<code>cd ~</code>
<code>cd -</code>	Return to previous directory	<code>cd -</code>
<code>tree -L 2</code>	Show directory tree 2 levels deep (may need install)	<code>tree -L 2 /opt</code>

Absolute vs relative paths

- Absolute: starts with `/` → `/etc/nginx/nginx.conf`
- Relative: from current directory → `../logs/app.log`
- Special: `.` = current directory, `..` = parent, `~` = home

Reading `ls -lah` output

```
-rw-r--r--  1 ubuntu ubuntu  2.1K Sep 20 10:14 config.yaml
drwxr-xr-x  3 root   root    4.0K Sep 18 08:02 nginx
lrwxrwxrwx  1 root   root      20 Sep 18 08:02 current ->
/opt/app/releases/v3
```

- First character: `-` file, `d` directory, `l` symlink.
- Next 9 characters: permissions for **owner** / **group** / **others**.
- Then: link count, owner, group, size, modified time, name.
- Hidden files start with a dot (`.env`, `.ssh`, `.bashrc`).

4. File & Directory Management

Command	What it does	Example
<code>mkdir -p <name></code>	Create folder (and missing parents)	<code>mkdir -p /opt/myapp/data</code>
<code>touch <file></code>	Create empty file / update timestamp	<code>touch .env</code>
<code>cp <src> <dest></code>	Copy a file	<code>cp config.yaml config.bak</code>
<code>cp -r <src> <dest></code>	Copy a directory recursively	<code>cp -r conf/ conf.bak/</code>
<code>cp -a</code>	Archive copy (preserves perms, owners, timestamps)	<code>cp -a /etc/nginx /backup/</code>
<code>mv <src> <dest></code>	Move or rename	<code>mv app.js server.js</code>
<code>rm <file></code>	Delete file	<code>rm old.log</code>
<code>rm -rf <path></code>	Force-delete directory and contents	<code>rm -rf old_build/</code>
<code>rmdir <dir></code>	Remove empty directory	<code>rmdir empty/</code>
<code>ln -s <target> <link></code>	Create a symbolic link	<code>ln -s /opt/app/v3 /opt/app/current</code>
<code>stat <file></code>	Detailed file metadata	<code>stat app.py</code>
<code>file <file></code>	Detect file type	<code>file server.bin</code>

⚠️ `rm -rf` is irreversible. There is no recycle bin. Never run it with an unchecked variable (e.g. `rm -rf "$DIR/"` when `$DIR` is empty expands to `rm -rf /`). Prefer `rm -rf -- "${DIR:?}/"` in scripts, and run `ls` on the path first.

Symlinks and the "current release" pattern — common in deployments:

```
bash
/opt/app/releases/v1
/opt/app/releases/v2
ln -sf /opt/app/releases/v2 /opt/app/current # atomic switch; easy rollback
```

Text editors

Editor	Notes
<code>nano file</code>	Beginner friendly. <code>(Ctrl+O)</code> then Enter = save, <code>(Ctrl+X)</code> = exit
<code>vim file</code> / <code>(vi file)</code>	Available everywhere. <code>(i)</code> insert, <code>(Esc)</code> then <code>(:wq)</code> save+quit, <code>(:q!)</code> quit without saving

Learn at least the basics of `(vi)` — it's often the only editor on minimal servers.

5. Viewing, Searching & Text Processing

5.1 Viewing files

Command	Purpose	Example
<code>cat <file></code>	Print entire file	<code>cat /etc/os-release</code>
<code>less <file></code>	Scrollable pager (<code>(/word)</code> search, <code>(q)</code> quit, <code>(G)</code> end)	<code>less /var/log/syslog</code>
<code>head -n 20 <file></code>	First 20 lines	<code>head -n 20 app.log</code>
<code>tail -n 50 <file></code>	Last 50 lines	<code>tail -n 50 app.log</code>
<code>tail -f <file></code>	Stream new lines live	<code>tail -f /var/log/syslog</code>
<code>tail -F <file></code>	Like <code>(-f)</code> but survives log rotation	<code>tail -F /var/log/nginx/access.log</code>
<code>wc -l <file></code>	Count lines	<code>wc -l access.log</code>

5.2 grep — search inside files

```
bash
```

```

grep "error" app.log           # lines containing "error"
grep -i "error" app.log        # case-insensitive
grep -n "error" app.log        # show line numbers
grep -v "debug" app.log        # invert: exclude matches
grep -r "DB_HOST" /etc/        # recursive search
grep -rn --include="*.yaml" "image:" . # only YAML files
grep -E "error|fatal|critical" app.log # extended regex (OR)
grep -c "404" access.log       # count matching lines
grep -A3 -B3 "Exception" app.log # 3 lines After / Before

```

5.3 find — search for files

bash

```

find /var/log -name "*.log"      # by name
find . -type f -size +100M      # files larger than 100 MB
find . -type f -mtime -1        # modified in last 24h
find . -type f -mtime +30       # older than 30 days
find /tmp -type f -mtime +7 -delete # delete files older than 7 days
find . -type f -name "*.sh" -exec chmod +x {} \; # run command on each result
find / -perm -4000 -type f 2>/dev/null # find SUID binaries (security audi

```

Also useful: `which <cmd>` (path of a command), `whereis <cmd>`, `locate <name>` (fast, indexed).

5.4 Text-processing power tools

bash

```

cut -d: -f1 /etc/passwd        # first field, ":" delimiter → use
sort access.log | uniq -c | sort -rn | head # top repeated lines
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head # top client I
awk '$9 == 500' access.log      # lines where field 9 is 500
sed 's/foo/bar/g' file.txt      # substitute (prints result)
sed -i 's/foo/bar/g' file.txt   # edit file in place
tr 'a-z' 'A-Z' < file.txt       # translate characters
xargs                           # build commands from stdin

```

Examples:

bash

```
# Count HTTP status codes in an nginx access log
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# Delete all .tmp files found
find . -name "*.tmp" -print0 | xargs -0 rm -f

# Save and display at once
dmesg | tee dmesg.txt | tail
```

Also: `(jq)` for JSON (`(curl -s api | jq '.items[].name')`), `(yq)` for YAML.

6. Pipes, Redirection & Shell Basics

Operator	Meaning	Example
<code> </code>	Pipe stdout of one command into another	<code>ps aux grep nginx</code>
<code>></code>	Redirect stdout to file (overwrite)	<code>ls > files.txt</code>
<code>>></code>	Redirect stdout to file (append)	<code>echo "hi" >> log.txt</code>
<code><</code>	Feed file to stdin	<code>sort < names.txt</code>
<code>2></code>	Redirect stderr	<code>cmd 2> errors.txt</code>
<code>2>&1</code>	Send stderr to the same place as stdout	<code>cmd > all.log 2>&1</code>
<code>&></code>	Bash shortcut: stdout+stderr	<code>cmd &> all.log</code>
<code>/dev/null</code>	Discard output	<code>cmd > /dev/null 2>&1</code>
<code>&&</code>	Run next only if previous succeeded	<code>make && make install</code>
<code> </code>	Run next only if previous failed	<code>cmd echo "failed"</code>
<code>;</code>	Run sequentially regardless	<code>cd /tmp; ls</code>
<code>\$(cmd)</code>	Command substitution	<code>echo "Today: \$(date)"</code>

Exit codes: every command returns a number; `0` = success, non-zero = failure. Check with `echo $?`.

Useful shell shortcuts

Shortcut	Action
<code>Tab</code>	Auto-complete
<code>Ctrl+C</code>	Interrupt running command
<code>Ctrl+Z</code>	Suspend job
<code>Ctrl+R</code>	Reverse-search history
<code>Ctrl+A</code> / <code>Ctrl+E</code>	Start / end of line
<code>Ctrl+L</code>	Clear screen
<code>!!</code>	Repeat last command (<code>sudo !!</code> re-runs with sudo)
<code>history</code>	Show command history

7. Users, Groups & sudo

```

bash

whoami                # current user
id                    # uid, gid, groups
who / w               # who is logged in
last                  # login history


sudo adduser deploy    # create user (Debian/Ubuntu, interactive)
sudo useradd -m -s /bin/bash deploy # create user (portable)
sudo passwd deploy     # set password
sudo usermod -aG docker deploy    # ADD to group (-a is crucial; without -a, user is removed from all groups)
sudo usermod -L deploy    # lock account
sudo userdel -r deploy    # delete user and home
sudo groupadd devs        # create group
groups deploy             # show user's groups

```

Key files

File	Contents
<code>/etc/passwd</code>	User accounts (name, UID, GID, home, shell)
<code>/etc/shadow</code>	Password hashes (root-readable only)
<code>/etc/group</code>	Groups and members
<code>/etc/sudoers</code>	sudo rules — edit only with <code>sudo visudo</code>
<code>/etc/sudoers.d/</code>	Drop-in sudo rule files (preferred)

sudo basics

bash

```
sudo command          # run a command as root
sudo -u www-data cmd  # run as another user
sudo -i               # root login shell
sudo -l               # list what you may run
```

Principle of least privilege: give users/service accounts only what they need. Avoid working as root.

Service accounts: create non-login system users for apps:

bash

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin apprunner
```

8. Permissions & Ownership

Every file has an **owner (user)**, a **group**, and permissions for **user / group / others**.

Permission	Letter	Number	On a file	On a directory
Read	<code>(r)</code>	4	View contents	List contents
Write	<code>(w)</code>	2	Modify contents	Create/delete/rename entries
Execute	<code>(x)</code>	1	Run as program	Enter (<code>(cd)</code>) / traverse

Add numbers for each class: `(rwx = 7)`, `(rw- = 6)`, `(r-x = 5)`, `(r-- = 4)`.

-rwxr-xr-- → owner: rwx (7) group: r-x (5) others: r-- (4) → 754

8.1 chmod

Command	Meaning
<code>chmod 755 script.sh</code>	Owner rwx; group & others r-x — standard for scripts/dirs
<code>chmod 644 config.json</code>	Owner rw; others read-only — standard for files
<code>chmod 600 key.pem</code>	Owner only — required for SSH private keys
<code>chmod 700 ~/.ssh</code>	Owner-only directory
<code>chmod +x run.sh</code>	Make executable
<code>chmod u+x,g-w,o-rwx file</code>	Symbolic mode: user +x, group -w, others none
<code>chmod -R 755 dir/</code>	Recursive (use with care)

8.2 chown / chgrp

bash

```
sudo chown ubuntu:ubuntu app.py          # owner:group
sudo chown -R www-data:www-data /var/www # recursive
sudo chown :devs shared.txt              # group only
sudo chgrp devs shared.txt
```

DevOps apps often run under service accounts (`www-data`, `nginx`, `apprunner`) — files they write to must be owned or writable by that account.

8.3 umask

Default permissions for new files are derived from `umask`. With `umask 022`: new files = `644`, new dirs = `755`. With `umask 077`: files = `600`, dirs = `700`.

8.4 Special permission bits

Bit	Numeric	Effect	Example
SUID	<code>4000</code>	Executable runs as the file's owner	<code>/usr/bin/passwd</code>
SGID	<code>2000</code>	Runs as group; on dirs, new files inherit the group	<code>chmod g+s shared/</code>
Sticky	<code>1000</code>	Only file owner can delete files in dir	<code>/tmp</code> (<code>drwxrwxrwt</code>)

SUID binaries are a classic privilege-escalation vector — audit them periodically.

8.5 ACLs (fine-grained)

```
bash

setfacl -m u:alice:rw file.txt
getfacl file.txt
```

8.6 Permission cheat

Mode	Typical use
600	Private keys, secrets, (.env)
640	Config readable by a service group
644	Normal files, public configs
700	Private directories ((~/ .ssh))
750	Directories shared with a group
755	Scripts, executables, public dirs
777	Avoid. World-writable = security problem

9. Package Management

Debian / Ubuntu ((apt))

```
bash

sudo apt update           # refresh package index
sudo apt upgrade -y       # upgrade installed packages
sudo apt install -y nginx curl git  # install
sudo apt remove nginx     # remove (keep config)
sudo apt purge nginx      # remove incl. config
sudo apt autoremove -y    # clean unused deps
apt search keyword
apt show nginx            # package details
dpkg -l | grep nginx      # installed packages
dpkg -L nginx             # files installed by a package
```

RHEL / Rocky / Fedora (`dnf`)

bash

```
sudo dnf check-update
sudo dnf install -y nginx
sudo dnf remove nginx
sudo dnf update -y
dnf search keyword
rpm -qa | grep nginx
rpm -ql nginx
```

Others

- Alpine: `apk add curl`
- Snap: `sudo snap install <name>`
- Always pin versions in production (`apt install nginx=1.24.0-1`) and prefer official repositories.

Never pipe unknown scripts into a root shell (`curl ... | sudo bash`) without reading them first.

10. Processes & Job Control

A **process** is a running program with a unique **PID**.

bash

<code>ps aux</code>	# all processes
<code>ps aux grep node</code>	# find your app (tip: use pgrep to avoid matching too many)
<code>pgrep -a nginx</code>	# PIDs + command lines by name
<code>ps -ef --forest</code>	# process tree
<code>pstree -p</code>	# tree view
<code>top / htop</code>	# live view (q to exit)

Stopping processes

Command	Signal	Meaning
<code>kill <PID></code>	<code>SIGTERM (15)</code>	Graceful shutdown request — process can clean up
<code>kill -9 <PID></code>	<code>SIGKILL (9)</code>	Force kill, no cleanup — last resort
<code>kill -HUP <PID></code>	<code>SIGHUP (1)</code>	Often "reload config" for daemons
<code>kill -STOP/-CONT</code>	—	Pause / resume
<code>pkill nginx</code>	—	Kill by name
<code>killall nginx</code>	—	Kill all by exact name

Always try `kill <PID>` first; use `kill -9` only if it refuses to exit (could leave lock files, corrupt data).

Foreground / background

```
bash
long_task &                # run in background
jobs                       # list shell jobs
fg %1 / bg %1              # bring to foreground / resume in background
nohup ./script.sh &        # keep running after logout
nohup ./script.sh > out.log 2>&1 &
```

For persistent interactive sessions on servers use `tmux` or `screen`:

```
bash
tmux new -s deploy         # start session
# Ctrl+B then D           → detach
tmux attach -t deploy      # reattach
```

Priority

```
bash
nice -n 10 ./heavy.sh      # start with lower priority
renice 5 -p <PID>          # change priority of running process
```

Zombie & orphan processes

- **Zombie:** finished, but parent hasn't collected exit status (state **Z**). Fix the parent; zombies cannot be killed directly.
 - **Orphan:** parent died; adopted by PID 1 (init/systemd).
-

11. systemd: Services, Timers & Logs

`systemd` (PID 1) manages services on almost all modern distros.

11.1 systemctl essentials

```
bash

sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx          # stop + start (brief downtime)
sudo systemctl reload nginx           # re-read config without dropping connections
sudo systemctl enable nginx           # start at boot
sudo systemctl enable --now nginx     # enable + start now
sudo systemctl disable nginx
systemctl status nginx                # state, recent logs, PID
systemctl is-active nginx
systemctl is-enabled nginx
systemctl list-units --type=service --state=running
systemctl --failed                    # failed units
sudo systemctl daemon-reload          # after editing unit files
```

11.2 Writing a custom service

`/etc/systemd/system/myapp.service`:

```
ini
```

```
[Unit]
Description=My Node.js App
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=aprunner
Group=aprunner
WorkingDirectory=/opt/myapp
EnvironmentFile=/opt/myapp/.env
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=5
LimitNOFILE=65535
# Hardening (optional but recommended)
NoNewPrivileges=true
ProtectSystem=full
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

bash

```
sudo systemctl daemon-reload
sudo systemctl enable --now myapp
systemctl status myapp
```

Use **drop-in overrides** instead of editing vendor units: `sudo systemctl edit nginx`.

11.3 journalctl (logs)

bash

```
journalctl -u nginx                # logs for a service
journalctl -u nginx -f              # follow live
journalctl -u nginx --since "1 hour ago"
journalctl -u nginx --since "2026-09-01" --until "2026-09-02"
journalctl -p err -b                # errors since this boot
journalctl -b -1                    # previous boot
journalctl --disk-usage
sudo journalctl --vacuum-time=7d    # trim old logs
```

12. System Monitoring & Performance

When an app is slow, out of memory, or failing, check server health in this order: **CPU** → **memory** → **disk** → **network**.

Area	Command	What you're checking
CPU & processes	<code>htop</code> / <code>top</code>	Real-time processes, per-core CPU, memory (press <code>q</code> to exit)
Uptime & load	<code>uptime</code>	Load averages (1, 5, 15 min)
RAM	<code>free -m</code> / <code>free -h</code>	Look at the available column, not "free"
Disk space	<code>df -h</code>	Free space per mounted filesystem
Directory size	<code>du -sh *</code> / <code>du -h --max-depth=1 /var</code>	Find what's eating space
Inodes	<code>df -i</code>	Out of inodes also means "disk full"
Processes	<code>ps aux grep <app></code>	Is it running? Get PID
Stop process	<code>kill <PID></code>	Graceful stop (<code>-9</code>) only if needed
Network ports	<code>ss -tulpn</code>	Which service listens on which port
Virtual memory/IO	<code>vmstat 1</code>	Swap, run queue, IO wait
Disk I/O	<code>iostat -xz 1</code> (sysstat)	Device utilization and latency
Open files	<code>lsof -i :80</code> / <code>lsof -p <PID></code>	What files/ports a process uses
Kernel messages	<code>dmesg -T tail</code>	OOM kills, hardware errors, disk problems

Reading the numbers

- **Load average:** compare to the number of CPU cores (`nproc`). Load 4.0 on a 4-core box ≈ fully utilized; 12.0 ≈ heavily overloaded.
- **`top` CPU line:** `us` user, `sy` system, `wa` IO wait (high = disk bottleneck), `st` steal (noisy VM neighbor).
- **Memory:** Linux uses spare RAM for cache — "free" being low is normal. **available** is what matters.

- **Swap in use + heavy swapping** (`si`/`so` in `vmstat`) = memory pressure.
- **OOM killer:** check `dmesg -T | grep -i "out of memory"` or `journalctl -k | grep -i oom`.

Handy one-liners

```
bash

# Top 10 memory consumers
ps aux --sort=-%mem | head -n 11

# Top 10 CPU consumers
ps aux --sort=-%cpu | head -n 11

# Biggest directories under /var
sudo du -h /var --max-depth=2 2>/dev/null | sort -rh | head -n 15

# Files > 500MB
sudo find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null

# Deleted-but-still-open files holding disk space
sudo lsof +L1
```

13. Disk, Storage & Filesystems

```
bash

lsblk                # block devices tree
lsblk -f             # with filesystem types and UUIDs
blkid                # UUIDs
sudo fdisk -l        # partition tables
df -hT              # usage + filesystem type
mount | column -t    # current mounts
```

Add and mount a new disk

```
bash

sudo fdisk /dev/sdb      # create partition (n, p, w) – or use partprobe
sudo mkfs.ext4 /dev/sdb1 # format (xfs: mkfs.xfs)
sudo mkdir -p /data
sudo mount /dev/sdb1 /data
```

Persist across reboots with `/etc/fstab` (use **UUID**, not device names):

```
UUID=1234-abcd-... /data ext4 defaults,nofail 0 2
```

```
bash
```

```
sudo mount -a # test fstab BEFORE rebooting
```

A bad `/etc/fstab` entry can prevent boot. Use `nofail` for non-critical mounts and always test with `mount -a`.

LVM (Logical Volume Manager) in brief

Allows resizing volumes online: **PV** (physical volume) → **VG** (volume group) → **LV** (logical volume).

```
bash
```

```
sudo pvcreate /dev/sdb
sudo vgcreate data_vg /dev/sdb
sudo lvcreate -L 20G -n data_lv data_vg
sudo mkfs.ext4 /dev/data_vg/data_lv
sudo lvextend -L +10G -r /dev/data_vg/data_lv # grow LV and filesystem
```

Swap

```
bash
```

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
swapon --show
```

Disk-full emergency checklist

1. `df -h` → which filesystem?
 2. `du -h --max-depth=1 /` (drill down) → what's large? Usually `/var/log`, Docker data (`/var/lib/docker`), caches.
 3. `docker system df` / `docker system prune` (if Docker).
 4. `sudo journalctl --vacuum-size=200M`, `sudo apt clean`.
 5. `df -i` → inode exhaustion? (many tiny files)
 6. `sudo lsof +L1` → deleted files still held open (restart the process).
-

14. Networking

14.1 Inspect

```
bash

ip a                # IP addresses (replaces ifconfig)
ip route            # routing table
ip link             # interfaces
hostname -I         # IPs of this host
ss -tulpn           # listening ports + owning process
ss -s               # socket summary
cat /etc/resolv.conf # DNS resolvers
cat /etc/hosts       # local name overrides
```

ss flags: **(-t)** TCP, **(-u)** UDP, **(-l)** listening, **(-p)** process, **(-n)** numeric.

14.2 Test connectivity

```
bash

ping -c 4 8.8.8.8      # ICMP reachability
ping -c 4 google.com   # reachability + DNS
traceroute google.com  # path (or tracepath / mtr)
mtr google.com         # live traceroute + loss stats
nc -zv host 443        # test if a TCP port is open
telnet host 25         # (legacy) port check
```

14.3 DNS

```
bash

dig example.com        # full DNS answer
dig +short example.com
dig example.com MX
dig @8.8.8.8 example.com # query a specific resolver
nslookup example.com
resolvectl status      # systemd-resolved info
```

14.4 Common ports

Port	Service	Port	Service
22	SSH	3306	MySQL
25	SMTP	5432	PostgreSQL
53	DNS	6379	Redis
80	HTTP	27017	MongoDB
443	HTTPS	6443	Kubernetes API
3000/8000/8080	Common app ports	2379-2380	etcd

14.5 Static IP / hostname

```
bash

sudo hostnamectl set-hostname web-01
# Ubuntu uses Netplan: /etc/netplan/*.yaml → sudo netplan apply
```

14.6 Packet capture

```
bash

sudo tcpdump -i eth0 port 80 -n
sudo tcpdump -i any host 10.0.0.5 -w capture.pcap # open in Wireshark
```

15. Firewall (UFW, firewalld, iptables)

15.1 UFW — Uncomplicated Firewall (Ubuntu/Debian)

```
bash
```

```
# 1. Check status
sudo ufw status verbose

# 2. ALWAYS allow SSH first so you don't lock yourself out!
sudo ufw allow 22/tcp

# 3. Allow web traffic
sudo ufw allow 80/tcp          # HTTP
sudo ufw allow 443/tcp        # HTTPS
sudo ufw allow 3000/tcp       # example custom app port

# 4. Enable
sudo ufw enable

# 5. Delete a rule
sudo ufw delete allow 3000/tcp
```

Useful extras:

```
bash

sudo ufw default deny incoming      # default policy (recommended)
sudo ufw default allow outgoing
sudo ufw allow from 203.0.113.10 to any port 22 proto tcp  # restrict SSH to o
sudo ufw allow 5432/tcp from 10.0.0.0/24                  # DB only from priv
sudo ufw limit 22/tcp                                     # rate-limit SSH (anti brute-force)
sudo ufw status numbered
sudo ufw delete 3                                         # delete by rule number
sudo ufw reload
sudo ufw disable
```

15.2 firewalld (RHEL family)

```
bash

sudo systemctl enable --now firewalld
sudo firewall-cmd --state
sudo firewall-cmd --list-all
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload
```

15.3 iptables / nftables (what's under the hood)

UFW and firewalld are front-ends. Kubernetes (kube-proxy), Docker, and CNI plugins also write rules.

```
bash
```

```
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
sudo nft list ruleset
```

Cloud note: AWS Security Groups / Azure NSGs / GCP firewall rules filter traffic **before** it reaches the host. If a port still isn't reachable after fixing UFW, check the cloud-level rules too.

16. SSH & Remote Access

16.1 Connect

```
bash
```

```
ssh ubuntu@203.0.113.10
ssh -i ~/.ssh/mykey.pem ubuntu@203.0.113.10
ssh -p 2222 user@host # non-default port
```

16.2 Key-based authentication

```
bash
```

```
ssh-keygen -t ed25519 -C "safdar@laptop" # generate keypair (modern, recommended)
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host # install public key on server
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519 ~/.ssh/authorized_keys
```

- Private key (`id_ed25519`): **never share**; must be `chmod 600` or SSH refuses it.
- Public key (`.pub`): goes in the server's `~/.ssh/authorized_keys`.

16.3 Client config (`~/.ssh/config`)

```
Host prod-web
  HostName 203.0.113.10
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
  Port 22
  ServerAliveInterval 60
```

Then simply: `ssh prod-web`.

16.4 File transfer

```
bash

scp file.txt user@host:/tmp/          # copy to remote
scp user@host:/var/log/app.log .      # copy from remote
scp -r ./dist user@host:/var/www/     # directory
rsync -avz --progress ./dist/ user@host:/var/www/app/ # efficient increment
rsync -avz --delete ./dist/ user@host:/var/www/app/   # mirror (deletes ext
```

16.5 Tunnels & jump hosts

```
bash

ssh -L 8080:localhost:80 user@host    # local forward: localhost:8080 →
ssh -R 9000:localhost:3000 user@host  # remote forward
ssh -D 1080 user@host                 # SOCKS proxy
ssh -J bastion@1.2.3.4 ubuntu@10.0.1.20 # via bastion (jump host)
```

16.6 Harden `sshd` (`/etc/ssh/sshd_config`)

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers deploy ubuntu
```

```
bash

sudo sshd -t          # validate config BEFORE restarting
sudo systemctl reload ssh # service is "sshd" on RHEL
```

Keep an existing SSH session open while testing changes so you can't lock yourself out.
Consider `fail2ban` to ban brute-force sources.

17. cURL & API Testing

```
bash
```

```
# Basic GET
curl https://example.com

# Headers/status only (HEAD request)
curl -I https://example.com

# Verbose: shows DNS, TLS handshake, request & response headers
curl -v https://example.com

# Test a local service
curl http://localhost:8000/health

# POST JSON
curl -X POST http://localhost:8000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "role": "DevOps"}'

# Follow redirects (-L) and save with remote filename (-O)
curl -LO https://github.com/example/releases/download/v1.0/app.tar.gz
```

Note: `(-L)` follows redirects and `(-O)` writes to a file named after the remote file. Add `(-s)` for silent mode (hide progress meter) and `(-S)` to still show errors.

More useful flags

Flag	Purpose
<code>-s</code> / <code>-sS</code>	Silent / silent but show errors
<code>-o file</code>	Save output to a chosen filename
<code>-H "Header: value"</code>	Add header (e.g. auth)
<code>-u user:pass</code>	Basic auth
<code>-k</code>	Skip TLS verification (testing only!)
<code>--max-time 10</code>	Total timeout
<code>--connect-timeout 5</code>	Connection timeout
<code>--retry 3</code>	Retry on transient errors
<code>-w "%{http_code}\n" -o /dev/null -s URL</code>	Print only the status code
<code>--resolve host:443:1.2.3.4</code>	Force DNS for testing
<code>-f</code>	Fail (non-zero exit) on HTTP errors — great for scripts

bash

```
# Health-check in a script
if curl -fsS --max-time 5 http://localhost:8000/health > /dev/null; then
    echo "UP"
else
    echo "DOWN"
fi

# Auth header
curl -H "Authorization: Bearer $TOKEN" https://api.example.com/v1/me

# Timing breakdown
curl -o /dev/null -s -w "dns:%{time_namelookup} connect:%{time_connect} ttfb:%{time_total}" https://api.example.com/v1/me
```

`wget URL` is another downloader (`wget -c` resumes).

HTTP status code quick reference

Code	Meaning
200 / 201 / 204	OK / Created / No Content
301 / 302	Redirect (permanent / temporary)
400 / 401 / 403 / 404	Bad request / Unauthenticated / Forbidden / Not found
429	Rate limited
500 / 502 / 503 / 504	Server error / Bad gateway / Unavailable / Gateway timeout

18. Archiving & Compression

bash

```
tar -czvf backup.tar.gz /opt/myapp      # create gzip archive
tar -xzvf backup.tar.gz                 # extract
tar -xzvf backup.tar.gz -C /restore/     # extract to a directory
tar -tzvf backup.tar.gz                 # list contents
tar -cJf backup.tar.xz dir/              # xz (smaller, slower)
gzip file.log / gunzip file.log.gz
zip -r site.zip site/ ; unzip site.zip
zcat file.log.gz | grep error            # read compressed logs
```

Flags: **(c)** create, **(x)** extract, **(t)** list, **(z)** gzip, **(v)** verbose, **(f)** file.

19. Scheduling: cron & systemd timers

19.1 cron

bash

```
crontab -e          # edit current user's crontab
crontab -l           # list
sudo crontab -u user -l
```

Format: **(minute hour day-of-month month day-of-week command)**

```

* * * * * command
| | | | | └─ day of week (0-7, Sun=0/7)
| | | └─── month (1-12)
| | └───── day of month (1-31)
| └──────── hour (0-23)
└────────── minute (0-59)

```

Examples:

```

*/5 * * * * /opt/scripts/healthcheck.sh # every 5 minutes
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1 # daily
02:00
0 0 * * 0 /opt/scripts/weekly.sh # Sunday midnight
@reboot /opt/scripts/on-boot.sh

```

Tips: use **absolute paths** (cron has a minimal `(PATH)`), redirect output to a log, and use `(flock)` to prevent overlapping runs: `(flock -n /tmp/job.lock /opt/job.sh)`.

19.2 systemd timers (modern alternative)

`(backup.service)` + `(backup.timer)`:

```

ini

# /etc/systemd/system/backup.timer
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target

```

```

bash

sudo systemctl enable --now backup.timer
systemctl list-timers

```

Benefits: logs in journald, dependency handling, catch-up with `(Persistent=true)`.

20. Environment Variables & Config Files

```
bash

printenv                # all variables
echo $PATH
export APP_ENV=production # available to child processes (current shell o
unset APP_ENV
env VAR=value command   # set for a single command
```

Persisting variables

File	Scope
<code>~/.bashrc</code>	Interactive non-login shells for one user
<code>~/.profile</code> / <code>~/.bash_profile</code>	Login shells for one user
<code>/etc/environment</code>	System-wide (simple <code>KEY=value</code>)
<code>/etc/profile.d/*.sh</code>	System-wide login scripts
systemd <code>EnvironmentFile=</code>	Per-service variables

```
bash

source ~/.bashrc           # reload without logging out
```

Aliases

```
bash

alias ll='ls -lah'
alias k='kubectl'
```

Secrets handling

- Keep `.env` files at `chmod 600`, owned by the service user; never commit them to Git.
- Prefer a secrets manager (Vault, AWS Secrets Manager, Kubernetes Secrets/External Secrets) over plain files in production.
- Avoid putting secrets in command-line arguments (visible via `ps`) or shell history.

21. Bash Scripting for Automation

21.1 Template

```
bash

#!/usr/bin/env bash
set -euo pipefail          # exit on error, unset vars are errors, fail on pipe
IFS=$'\n\t'

LOG_FILE="/var/log/deploy.log"
APP_DIR="/opt/myapp"

log() { echo "[$(date '+%F %T')] $*" | tee -a "$LOG_FILE"; }

usage() {
    echo "Usage: $0 <version>"
    exit 1
}

[[ $# -eq 1 ]] || usage
VERSION="$1"

log "Deploying version ${VERSION}"
cd "$APP_DIR"
git fetch --tags
git checkout "$VERSION"
npm ci --omit=dev
sudo systemctl restart myapp

# Verify
sleep 3
if curl -fsS --max-time 5 http://localhost:3000/health >/dev/null; then
    log "Deployment OK"
else
    log "Health check FAILED"
    exit 1
fi
```

Make executable and run: `chmod +x deploy.sh && ./deploy.sh v1.2.0`

21.2 Essentials

```
bash
```

```

# Variables
NAME="Safdar"; echo "Hello, $NAME"; echo "${NAME}_backup"

# Conditionals
if [[ -f /etc/nginx/nginx.conf ]]; then echo "exists"; fi
# Test flags: -f file, -d dir, -e exists, -r readable, -x executable, -z empty

# Numeric comparison: -eq -ne -gt -lt -ge -le
if [[ $COUNT -gt 5 ]]; then echo "many"; fi

# Loops
for host in web1 web2 web3; do ssh "$host" uptime; done
for f in *.log; do gzip "$f"; done
while read -r line; do echo "$line"; done < file.txt

# Functions & exit codes
check() { systemctl is-active --quiet "$1"; }
check nginx && echo "nginx up" || echo "nginx down"

# Arguments
# $0 script name $1..$9 args $# arg count @$ all args $? last exit code

# Default values
PORT="${PORT:-8080}"

# case
case "$1" in
  start) echo "starting" ;;
  stop)  echo "stopping" ;;
  *)     echo "usage: $0 {start|stop}" ;;
esac

# trap for cleanup
trap 'rm -f /tmp/lockfile' EXIT

```

21.3 Scripting best practices

- Always quote variables: `"$VAR"`.
- Use `set -euo pipefail` for safety.
- Make scripts **idempotent** (safe to run twice).
- Lint with `shellcheck script.sh`.
- Log with timestamps; return meaningful exit codes.
- Don't hardcode secrets.

22. Logging & Log Management

Common log locations

Path	Contents
<code>/var/log/syslog</code> (Debian) / <code>/var/log/messages</code> (RHEL)	General system messages
<code>/var/log/auth.log</code> (Debian) / <code>/var/log/secure</code> (RHEL)	Authentication, sudo, SSH logins
<code>/var/log/kern.log</code> / <code>dmesg</code>	Kernel messages
<code>/var/log/nginx/access.log</code> , <code>error.log</code>	Nginx
<code>/var/log/apt/history.log</code>	Package operations
<code>journalctl</code>	systemd journal (binary)

Useful investigations

```
bash

sudo tail -f /var/log/auth.log # live auth events
sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort |
sudo grep -i "error" /var/log/nginx/error.log | tail -n 50
last -a | head ; lastb | head # successful logins
```

Log rotation (`logrotate`) — `/etc/logrotate.d/myapp`:

```
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

```
bash

sudo logrotate -d /etc/logrotate.d/myapp # dry run / debug
sudo logrotate -f /etc/logrotate.d/myapp # force
```

Centralized logging: at scale ship logs to a central stack — ELK/OpenSearch, Loki + Grafana, Datadog, CloudWatch — using agents like Promtail, Fluent Bit, Filebeat.

23. Security Hardening Checklist

- ☐ Keep system updated: `(sudo apt update && sudo apt upgrade)` (consider `(unattended-upgrades)` for security patches).
 - ☐ Disable root SSH login and password authentication; use keys only.
 - ☐ Use a non-root user with `(sudo)`; apply least privilege.
 - ☐ Enable firewall; default **deny incoming**; open only required ports.
 - ☐ Restrict SSH to known IPs or a bastion/VPN where possible.
 - ☐ Install `(fail2ban)` for brute-force protection.
 - ☐ Remove unused packages and stop unused services (`(ss -tulpn)` to audit listening ports).
 - ☐ Set correct file permissions (`(600)` for secrets/keys, avoid `(777)`).
 - ☐ Audit SUID/SGID binaries: `(find / -perm -4000 -type f 2>/dev/null)`.
 - ☐ Use separate service accounts per application.
 - ☐ Enable and review auditing/logging (`(auditd)`, `(/var/log/auth.log)`).
 - ☐ Use SELinux (RHEL) or AppArmor (Ubuntu) — keep them **enabled** (`(getenforce)`, `(aa-status)`).
 - ☐ Time sync via `(chrony)` / `(systemd-timesyncd)` (`(timedatectl)`) — critical for TLS and log correlation.
 - ☐ Encrypt data in transit (TLS) and at rest where required (LUKS).
 - ☐ Take backups and **test restores**.
 - ☐ Scan: `(lynis audit system)`, `(nmap)` (only on systems you own/are authorized to test), CVE scanners for packages and images.
-

24. Linux Internals Behind Containers & Kubernetes

Containers are not VMs — they are regular Linux processes isolated by kernel features:

Feature	Role
Namespaces	Isolate what a process can see: <code>(pid)</code> , <code>(net)</code> , <code>(mnt)</code> , <code>(uts)</code> (hostname), <code>(ipc)</code> , <code>(user)</code> , <code>(cgroup)</code>
cgroups	Limit/measure what a process can use: CPU, memory, IO, pids
Union/overlay filesystems	Layered image filesystems (<code>(overlayfs)</code>)
Capabilities / seccomp	Restrict privileged syscalls
iptables / nftables / eBPF	Container & Kubernetes networking, kube-proxy, CNI

```
bash
```

```
lsns                                # list namespaces
sudo nsenter -t <PID> -n ip a        # enter a container's network namespace
cat /sys/fs/cgroup/...              # cgroup limits
ps -o pid,ns -p <PID>
```

Node prerequisites (Kubernetes / kubeadm)

```
bash
```

```
sudo swapoff -a                    # kubelet traditionally requires
# comment out swap in /etc/fstab to persist
sudo modprobe overlay br_netfilter
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

Kernel tuning with sysctl

```
bash
```

```
sysctl -a | grep somaxconn
sudo sysctl -w net.core.somaxconn=1024          # temporary
echo "net.core.somaxconn=1024" | sudo tee /etc/sysctl.d/99-custom.conf # pers
ulimit -n                                       # open-file limit (set via /etc
```

Relevant when debugging: *"too many open files"*, *conntrack table full*, *OOMKilled* (cgroup memory limit hit), and pods that can't reach each other (firewall/iptables/CNI).

25. Troubleshooting Playbook

25.1 The golden order (from the cheat sheet)

When something fails, check in **this order**:

1. **Is the process running?** → `ps aux | grep <app>` / `systemctl status <svc>`
2. **Is it listening on the right port?** → `ss -tulpn`
3. **Is the firewall allowing that port?** → `sudo ufw status`
4. **Can you reach it locally with cURL?** → `curl -I localhost:<port>`

Then extend outward: **remote cURL** → **DNS** → **cloud security group** / **load balancer** → **logs**.

25.2 Scenario guides

"Site is down / connection refused"

```
bash

systemctl status nginx          # running?
sudo ss -tulpn | grep :80      # listening?
curl -I localhost              # works locally?
sudo ufw status                 # firewall?
sudo nginx -t                  # config valid?
journalctl -u nginx -n 100 --no-pager
```

Connection refused → nothing is listening (service down / wrong port/bind address).

Timeout → blocked by firewall/security group or routing issue. *Bound to 127.0.0.1 only* → remote clients can't connect; bind to `0.0.0.0` or a proper interface.

"Server is slow"

```
bash
```

```

uptime ; nproc                # load vs cores
top                            # CPU hog? wa (iowait) high?
free -h ; vmstat 1 5          # memory/swap pressure
df -h ; iostat -xz 1 3        # disk full / IO saturated
ss -s                          # too many connections?
dmesg -T | tail                # kernel errors / OOM

```

"Disk full" → see [Section 13 checklist](#).

"Service won't start"

```

bash

systemctl status myapp -l
journalctl -xeu myapp
sudo systemd-analyze verify /etc/systemd/system/myapp.service
# Common causes: wrong path in ExecStart, permissions, missing env file, port a
sudo ss -tulpn | grep :3000

```

"Permission denied"

```

bash

ls -l file ; id                # compare owner/group vs your user
namei -l /path/to/file         # check every directory in the path (needs +x)
getenforce ; sudo ausearch -m avc -ts recent # SELinux denials (RHEL)

```

"Cannot SSH in"

```

bash

ssh -vvv user@host            # verbose handshake
# Check: correct key & chmod 600, user name, port, security group/firewall, ssh

```

"Process keeps dying" → `dmesg -T | grep -i -E "killed process|oom"`; check `journalctl -u app`; check `Restart=` policy and resource limits.

"Name not resolving" → `dig example.com`, `cat /etc/resolv.conf`, `resolvectl status`, try `dig @8.8.8.8`.

25.3 Mindset

1. Define the symptom precisely.
 2. Check what changed recently.
 3. Form a hypothesis.
 4. Test **one** thing at a time.
 5. Read the logs.
 6. Document the fix / write a runbook.
-

26. Time, Date & NTP

Accurate time is critical: TLS validation, log correlation, Kerberos, cron, distributed databases, and Kubernetes certificates all depend on it.

```
bash

date                                # current date/time
date -u                             # UTC
timedatectl                         # time, timezone, NTP sync status
sudo timedatectl set-timezone UTC    # servers: UTC is the best practice
timedatectl list-timezones | grep Karachi
sudo timedatectl set-ntp true        # enable network time sync
```

NTP clients

Tool	Notes
<code>systemd-timesyncd</code>	Lightweight SNTP client (default on Ubuntu)
<code>chrony</code>	Full-featured, preferred on RHEL and for servers needing accuracy

```
bash

sudo apt install -y chrony
chronyc tracking                # offset from reference
chronyc sources -v              # NTP servers in use
```

Useful date formats for scripts and backups:

```
bash

date +%F                          # 2026-10-03
date +%F_%H-%M-%S                 # 2026-10-03_14-30-05
date -d "yesterday" +%F
date +%s                          # epoch seconds
```

27. TLS/SSL & Certificates

27.1 Concepts

- **Private key:** secret; never share (`chmod 600`).
- **CSR:** certificate signing request sent to a Certificate Authority (CA).
- **Certificate:** public key + identity, signed by a CA.

- **Chain:** server cert → intermediate(s) → root CA. Servers must send the full chain.

27.2 openssl essentials

```
bash

# Generate a private key + self-signed cert (testing only)
openssl req -x509 -newkey rsa:4096 -sha256 -days 365 -nodes \
  -keyout server.key -out server.crt -subj "/CN=myapp.local"

# Generate key + CSR for a CA
openssl req -new -newkey rsa:2048 -nodes -keyout server.key -out server.csr -su

# Inspect a certificate file
openssl x509 -in server.crt -text -noout
openssl x509 -in server.crt -noout -dates -subject -issuer

# Inspect a live server's certificate
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev
  | openssl x509 -noout -dates -subject -issuer

# Days until expiry check (alerting script)
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev
  | openssl x509 -noout -enddate

# Verify a key matches a cert (hashes must be identical)
openssl x509 -noout -modulus -in server.crt | openssl md5
openssl rsa -noout -modulus -in server.key | openssl md5
```

27.3 Let's Encrypt with certbot

```
bash

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com
sudo certbot renew --dry-run          # test auto-renewal
systemctl list-timers | grep certbot # renewal timer
```

Requirements: DNS pointing to the server and port 80/443 reachable. Certificates last 90 days, so automate renewal and monitor expiry.

27.4 Common TLS problems

Symptom	Likely cause
<code>certificate has expired</code>	Renewal failed — check certbot/timer logs
<code>unable to get local issuer certificate</code>	Missing intermediate in the chain
<code>hostname mismatch</code>	Cert CN/SAN doesn't include the requested name
<code>certificate is not yet valid</code>	Client or server clock wrong (see Section 26)

`curl -v https://host` and `openssl s_client -connect host:443 -showcerts` are the fastest diagnostics.

28. Nginx & Reverse Proxy Essentials

Nginx is a web server, reverse proxy and load balancer — one of the most common services you'll operate.

```
bash

sudo apt install -y nginx
sudo nginx -t                # ALWAYS test config before reload
sudo systemctl reload nginx
```

Key paths (Debian/Ubuntu)

Path	Purpose
<code>/etc/nginx/nginx.conf</code>	Main config
<code>/etc/nginx/sites-available/</code>	Site configs (all)
<code>/etc/nginx/sites-enabled/</code>	Symlinks to active sites
<code>/var/log/nginx/access.log</code> , <code>error.log</code>	Logs
<code>/var/www/html</code>	Default web root

Reverse proxy to an app on port 3000

```
nginx
```

```

server {
    listen 80;
    server_name example.com www.example.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade; # WebSocket support
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 60s;
    }
}

```

bash

```

sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

Load balancing

nginx

```

upstream backend {
    least_conn;
    server 10.0.0.11:3000;
    server 10.0.0.12:3000;
    server 10.0.0.13:3000 backup;
}
server { location / { proxy_pass http://backend; } }

```

Troubleshooting Nginx

Error	Meaning / Fix
502 Bad Gateway	Upstream app is down or wrong port — check <code>ss -tulpn</code> , app logs
504 Gateway Timeout	Upstream too slow — raise <code>proxy_read_timeout</code> , check app performance
403 Forbidden	File permissions or missing index; web user (<code>www-data</code>) can't read path
404	Wrong <code>root</code> / <code>location</code> or file missing
413 Request Entity Too Large	Raise <code>client_max_body_size</code>
<code>bind() failed (98: Address already in use)</code>	Another process holds the port: <code>sudo ss -tulpn grep :80</code>

29. Git for DevOps

bash

```
git clone <url>           # copy a repo
git status                # working tree state
git add -p                # stage interactively
git commit -m "feat: add health check"
git log --oneline --graph --decorate -n 20
git diff                  # unstaged changes
git diff --staged         # staged changes
git pull --rebase         # update with linear history
git push origin main
```

Branching & tagging

bash

```
git switch -c feature/login # create + switch branch
git switch main
git merge feature/login     # or rebase
git branch -d feature/login
git tag -a v1.2.0 -m "Release 1.2.0" && git push origin v1.2.0
```

Undo / recover

bash

```
git restore file                # discard unstaged changes to file
git restore --staged file       # unstage
git revert <commit>             # safe undo: new commit (use on shared branch)
git reset --hard <commit>       # DANGEROUS: rewrites working tree; local-only
git stash ; git stash pop       # shelve and restore changes
git reflog                      # find "lost" commits
```

Handy

bash

```
git bisect start ; git bisect bad ; git bisect good <commit>  # find the commit
git blame file                # who changed each line
git cherry-pick <commit>      # apply a single commit elsewhere
git config --global user.name "Name" ; git config --global user.email "me@example.com"
```

Git on servers

- Use **deploy keys** (read-only SSH keys per repo) rather than personal credentials.
- **Never commit secrets.** Add `.env`, `*.pem`, `*.key` to `.gitignore`. If committed, rotate the secret — deleting the commit isn't enough. Use scanners such as `gitleaks`.
- Common branching models: trunk-based development, GitFlow. Use protected branches + PR reviews.

30. Docker on Linux

bash

```
# Install (Ubuntu quick path)
curl -fsSL https://get.docker.com | sudo sh      # review the script first in
sudo usermod -aG docker $USER && newgrp docker    # run docker without sudo (no
sudo systemctl enable --now docker
docker version ; docker info
```

Images & containers

bash

```

docker pull nginx:1.27
docker run -d --name web -p 8080:80 nginx:1.27      # detached, publish host 8080
docker run -it --rm ubuntu:24.04 bash              # throwaway interactive shell
docker ps ; docker ps -a                          # running / all
docker logs -f web                                # follow logs
docker exec -it web sh                             # shell into running container
docker stop web ; docker rm web
docker images ; docker rmi <image>
docker inspect web                                # JSON details (IP, mounts,
docker stats                                       # live CPU/mem per container

```

Volumes, networks, env

```

bash

docker run -d -v mydata:/var/lib/postgresql/data -e POSTGRES_PASSWORD=secret postgres
docker run -v $(pwd)/conf:/etc/app:ro myimage      # bind mount, read-only
docker volume ls ; docker network ls
docker network create appnet
docker run --network appnet --name db postgres:16

```

Dockerfile example (multi-stage, non-root)

```

dockerfile

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
EXPOSE 3000
HEALTHCHECK CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]

```

```

bash

```

```
docker build -t myapp:1.0 .
docker compose up -d ; docker compose logs -f ; docker compose down
```

Cleanup & disk usage

```
bash

docker system df
docker system prune -f           # stopped containers, dangling images, u
docker image prune -a            # all unused images (careful)
docker volume prune              # unused volumes (DATA LOSS risk)
```

Best practices: pin image tags (avoid `latest`), run as non-root, use small base images, keep secrets out of images, scan images (`trivy image myapp:1.0`), set resource limits (`--memory`), (`--cpus`), and log to stdout/stderr.

Docker publishes ports by writing iptables rules, which can **bypass UFW**. A port published with `-p` may be reachable even if UFW doesn't allow it. Bind to localhost (`-p 127.0.0.1:8080:80`) or manage the `DOCKER-USER` chain.

31. Kubernetes Quick Ops (kubectl)

Context: Linux fundamentals underpin every node (Section 24). Day-to-day cluster operations:

```
bash
```

```

kubect1 config get-contexts ; kubect1 config use-context <ctx>
kubect1 get nodes -o wide
kubect1 get pods -A # all namespaces
kubect1 get pods -n prod -o wide
kubect1 describe pod <pod> -n prod # events show why it's fa
kubect1 logs <pod> -n prod -f --tail=100
kubect1 logs <pod> -c <container> --previous # logs of the crashed ins
kubect1 exec -it <pod> -n prod -- sh
kubect1 apply -f deployment.yaml
kubect1 delete -f deployment.yaml
kubect1 rollout status deploy/myapp
kubect1 rollout undo deploy/myapp # rollback
kubect1 scale deploy/myapp --replicas=5
kubect1 top nodes ; kubect1 top pods # needs metrics-server
kubect1 port-forward svc/myapp 8080:80
kubect1 get events --sort-by=.lastTimestamp -A
kubect1 explain pod.spec.containers # built-in docs
kubect1 get pod <pod> -o yaml

```

Pod status → typical cause

Status	Likely cause	First step
Pending	No node fits (resources, taints, PVC unbound)	kubect1 describe pod → Events
ImagePullBackOff	Wrong image/tag or missing registry credentials	Check image name, imagePullSecrets
CrashLoopBackOff	App crashes on start	kubect1 logs --previous, check config/env
OOMKilled	Exceeded memory limit	Raise limit / fix leak; dmesg on node
Evicted	Node pressure (disk/memory)	kubect1 describe node
Running but not ready	Readiness probe failing	Check probe path/port

Node-level (Linux) checks when the cluster misbehaves

```
bash
```

```
systemctl status kubelet containerd
journalctl -u kubelet -f
sudo crictl ps ; sudo crictl logs <id>
free -h ; df -h ; swapon --show
```

32. Backup & Recovery

3-2-1 rule: keep 3 copies of data, on 2 different media, with 1 off-site. A backup you haven't test-restored is not a backup.

32.1 Files

```
bash

# Timestamped archive
tar -czf /backup/app_$(date +%F).tar.gz /opt/myapp

# Incremental mirror with rsync (hard-link snapshots)
rsync -a --delete --link-dest=/backup/latest /opt/myapp/ /backup/$(date +%F)/
ln -sfn /backup/$(date +%F) /backup/latest

# Off-site
rsync -avz -e ssh /backup/ backup@remote:/srv/backups/
```

32.2 Databases

```
bash

pg_dump -U postgres -Fc mydb > mydb_$(date +%F).dump      # PostgreSQL (custom format)
pg_restore -U postgres -d mydb_restore mydb_2026-10-03.dump
mysqldump --single-transaction -u root -p mydb | gzip > mydb_$(date +%F).sql.gz
```

Copying live database files with `cp` can produce a corrupt backup — use the dump tools or consistent snapshots.

32.3 Retention script

```
bash

#!/usr/bin/env bash
set -euo pipefail
DEST=/backup
tar -czf "$DEST/app_$(date +%F).tar.gz" /opt/myapp
find "$DEST" -name 'app_*.tar.gz' -mtime +7 -delete      # keep 7 days
```

32.4 Other tools & ideas

- `restic` / `borg`: encrypted, deduplicated backups (local, S3, SFTP).
- LVM or cloud snapshots for point-in-time volume copies.
- Define **RPO** (how much data loss is acceptable) and **RTO** (how fast you must recover).
- Monitor backup jobs and alert on failure; run periodic restore drills.

33. Advanced Debugging Tools

Tool	Use
<code>strace -f -p <PID></code>	Trace system calls of a running process (file, network, permission errors)
<code>strace -f -e trace=file cmd</code>	Show which files a command opens
<code>ltrace</code>	Library calls
<code>lsof -p <PID></code> / <code>lsof -i :80</code>	Open files and sockets
<code>ldd /usr/bin/app</code>	Shared libraries required (missing <code>.so</code> errors)
<code>perf top</code> / <code>perf record</code>	CPU profiling
<code>tcpdump</code> / <code>ngrep</code>	Packet inspection
<code>watch -n 2 'ss -s'</code>	Re-run a command every 2 seconds
<code>ncdu /var</code>	Interactive disk usage browser
<code>nmap -p- host</code>	Port scan (only systems you are authorized to test)
<code>dmesg -T</code>	Kernel ring buffer (OOM, disk, NIC issues)
<code>sar</code> (sysstat)	Historical CPU/memory/IO/network stats
<code>iotop</code>	Per-process disk IO
<code>nload</code> / <code>iftop</code>	Network bandwidth
<code>/proc/<PID>/</code>	Live process info (<code>status</code> , <code>limits</code> , <code>fd/</code> , <code>cmdline</code>)

Examples

bash

```
# Why can't the app start? See what it tries and fails to open
strace -f -e trace=file,openat ./myapp 2>&1 | grep -E "ENOENT|EACCES"

# What is a hung process doing?
sudo strace -p <PID>
sudo cat /proc/<PID>/stack          # kernel stack (if permitted)

# Count open file descriptors for a process
ls /proc/<PID>/fd | wc -l ; cat /proc/<PID>/limits | grep "open files"

# Find the process using a port
sudo ss -tulpn | grep :8080 ; sudo lsof -i :8080

# OOM history
journalctl -k | grep -i -E "out of memory|killed process"
```

Systematic approach: reproduce → read logs → check resources → trace syscalls/network → change one variable → verify → write it up.

34. SELinux & AppArmor

Mandatory Access Control (MAC) restricts what processes can do even if they run as root or files have permissive modes.

34.1 SELinux (RHEL, Rocky, Fedora)

bash

```
getenforce          # Enforcing | Permissive | Disabled
sudo setenforce 0   # temporary permissive (debugging only!)
sestatus
ls -Z /var/www/html # show security contexts
ps -eZ | grep nginx
sudo ausearch -m avc -ts recent # recent denials
sudo sealert -a /var/log/audit/audit.log
```

Common fixes (instead of disabling SELinux):

bash

```
# Wrong file context for a custom web root
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"
sudo restorecon -Rv /srv/site

# Allow nginx/httpd to make outbound network connections
sudo setsebool -P httpd_can_network_connect 1

# Allow a non-standard port
sudo semanage port -a -t http_port_t -p tcp 8081
```

34.2 AppArmor (Ubuntu, Debian, SUSE)

```
bash

sudo aa-status
sudo aa-complain /usr/sbin/nginx # log only
sudo aa-enforce /usr/sbin/nginx
journalctl -k | grep -i apparmor # denials
# Profiles live in /etc/apparmor.d/
```

Rule of thumb: when you see "Permission denied" but classic permissions look correct, suspect SELinux/AppArmor. Fix the context or policy — don't turn it off.

35. Boot Process, Targets & Recovery

Boot sequence: firmware (BIOS/UEFI) → bootloader (GRUB) → kernel + initramfs → systemd (PID 1) → targets/services → login.

```
bash

systemd-analyze # total boot time
systemd-analyze blame | head # slowest units
systemctl get-default # multi-user.target (CLI) or graphical.target
sudo systemctl set-default multi-user.target
uname -r ; ls /boot # running kernel; installed kernels
```

Targets (replace old runlevels)

Target	Old runlevel	Meaning
<code>poweroff.target</code>	0	Shutdown
<code>rescue.target</code>	1	Single-user maintenance
<code>multi-user.target</code>	3	Multi-user, no GUI
<code>graphical.target</code>	5	GUI
<code>reboot.target</code>	6	Reboot

Recovery scenarios

- **Forgot root password / broken config:** at GRUB press `(e)`, append `(systemd.unit=rescue.target)` (or `(init=/bin/bash)`) to the `(linux)` line, then `(Ctrl+X)`. Remount rw: `(mount -o remount,rw /)`.
- **Broken `/etc/fstab`:** boot into rescue mode, fix the entry, test with `(mount -a)`.
- **Boot loop after kernel update:** pick an older kernel in GRUB "Advanced options".
- **Rebuild GRUB:** `(sudo update-grub)` (Debian) / `(sudo grub2-mkconfig -o /boot/grub2/grub.cfg)` (RHEL).
- **Cloud VM unreachable:** attach the disk to a rescue instance, mount, fix, reattach.

```
bash

sudo reboot ; sudo shutdown -h +10 "Maintenance in 10 min" ; sudo shutdown -c
last reboot | head                                # reboot history
```

36. Infrastructure as Code & Configuration Management

Manual server changes drift and can't be reproduced. IaC makes infrastructure **versioned**, **reviewable**, **repeatable**.

Tool	Purpose
Terraform / OpenTofu	Provision cloud resources (VMs, networks, DBs) declaratively
Ansible	Configure servers over SSH (agentless), deploy apps, run ad-hoc tasks
cloud-init	First-boot VM configuration (users, packages, files)
Packer	Build golden machine images
Helm / Kustomize	Package and customize Kubernetes manifests

Ansible essentials

inventory.ini

ini

[web]

web1 ansible_host=203.0.113.10 ansible_user=ubuntu

web2 ansible_host=203.0.113.11 ansible_user=ubuntu

site.yml (idempotent playbook)

yaml

```
- hosts: web
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
    - name: Ensure nginx is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
    - name: Allow HTTP in UFW
      community.general.ufw:
        rule: allow
        port: "80"
        proto: tcp
```

bash

ansible all -i inventory.ini -m ping

ansible-playbook -i inventory.ini site.yml --check --diff # dry run

ansible-playbook -i inventory.ini site.yml

ansible-vault encrypt secrets.yml # encrypt secrets

Terraform workflow

bash

```
terraform init ; terraform fmt ; terraform validate
terraform plan -out tfplan          # review changes
terraform apply tfplan
terraform destroy                    # tear down (careful)
```

Keep state in a **remote backend** with locking (S3 + DynamoDB, Terraform Cloud, etc.); never commit state files or secrets.

cloud-init snippet

```
yaml

#cloud-config
users:
  - name: deploy
    groups: sudo
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-ed25519 AAAA... me@laptop
packages: [nginx, git, ufw]
runcmd:
  - ufw allow 22/tcp && ufw allow 80/tcp && ufw --force enable
```

37. CI/CD on Linux

A pipeline automates **build** → **test** → **scan** → **package** → **deploy**. Runners/agents are Linux machines or containers executing your shell commands, so everything above applies.

Typical stages

1. Checkout code ((`git`))
2. Install deps & build ((`npm ci`), (`pip install`), (`docker build`))
3. Test & lint ((`pytest`), (`shellcheck`), (`hadolint`))
4. Security scan ((`trivy`), (`gitleaks`), dependency audit)
5. Publish artifact/image to a registry
6. Deploy (SSH + systemd, (`kubectl`)/Helm, Ansible, Terraform)
7. Smoke test & notify ((`curl -fsS /health`))

GitHub Actions example

```
yaml
```

```

name: ci
on:
  push:
    branches: [main]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm test
      - run: docker build -t ghcr.io/${{ github.repository }}:${{ github.sha }}

```

Deploy via SSH (conceptual)

```

bash

ssh deploy@server 'cd /opt/myapp && git pull && npm ci --omit=dev && sudo systemctl restart myapp && curl -fsS --retry 5 --retry-delay 2 https://example.com/health'

```

Practices: store secrets in the CI secret store (not the repo), use least-privilege deploy users/keys, tag releases, make deployments atomic with rollback (symlinked releases or `kubectl rollout undo`), and deploy the same artifact to every environment.

38. Master Cheat Sheet

Navigation & files

```

pwd | ls -lah | cd path | cd .. | cd ~ | cd -
mkdir -p d | touch f | cp -r a b | mv a b | rm -rf d | ln -s t l
cat | less | head | tail -f | wc -l | nano | vim

```

Search & text

```

grep -rni "text" dir | find / -name "x" | awk | sed | sort | uniq -c | cut |
xargs | jq

```

Permissions

```

chmod 755 / 644 / 600 / +x      chown user:group file      chown -R user:group
dir

```

Users

```
id | whoami | adduser | usermod -aG grp user | passwd | sudo -l | visudo
```

Processes & services

```
ps aux | top/htop | kill PID | kill -9 PID | pgrep | nohup cmd &  
systemctl status|start|stop|restart|reload|enable|disable svc  
journalctl -u svc -f
```

Monitoring

```
uptime | free -h | df -h | du -sh * | vmstat 1 | iostat -xz 1 | ss -tulpn |  
lsof -i :PORT | dmesg -T
```

Network

```
ip a | ip route | ping | traceroute/mtr | dig | nc -zv host port | curl -I  
url | tcpdump
```

Firewall (UFW)

```
ufw status verbose | ufw allow 22/tcp | ufw allow 80/tcp | ufw enable | ufw  
delete allow PORT/tcp
```

SSH

```
ssh user@host | ssh-keygen -t ed25519 | ssh-copy-id | scp | rsync -avz | ssh  
-L/-R/-J
```

TLS / Docker / Kubernetes / Git

```
openssl s_client -connect host:443 | openssl x509 -noout -dates  
docker ps|logs -f|exec -it|compose up -d          kubect1  
get|describe|logs|rollout undo  
git status|log --oneline|pull --rebase|revert  
nginx -t && systemctl reload nginx                certbot renew --dry-run
```

Debugging

```
strace -f -p PID | lsof -i :PORT | ldd bin | dmesg -T | journalctl -xeu svc |  
watch -n2 cmd
```

Archives

```
tar -czvf out.tgz dir | tar -xzvf out.tgz | zip -r | unzip
```

Packages

```
apt update && apt upgrade | apt install pkg | dnf install pkg
```

39. Best Practices & Golden Rules

1. **Never work as root** — use a named user + `(sudo)` so actions are audited.
2. **Read before you run** — `(ls)`, `(cat)`, and dry-run flags (`(--dry-run)`, `(-n)`) before destructive commands.
3. **Back up before editing** configs: `(cp file file.bak.$(date +%F))`.
4. **Test config before reload**: `(nginx -t)`, `(sshd -t)`, `(visudo -c)`, `(mount -a)`.
5. **Don't lock yourself out**: allow SSH before enabling a firewall; keep a second session open.
6. **Prefer (reload) over (restart)** when supported.
7. **Graceful first**: `(kill)` (SIGTERM) before `(kill -9)`.
8. **Automate everything repeatable** — scripts, then Ansible/Terraform; keep them in Git.
9. **Make changes idempotent and reversible** (symlinked releases, versioned configs).
10. **Least privilege everywhere** — files, users, ports, network access.
11. **Log and monitor** — you can't fix what you can't see; alert on symptoms.
12. **Document runbooks** for recurring incidents.
13. **Treat servers as cattle, not pets** — prefer immutable infrastructure and reproducible builds.
14. **Keep systems patched**, and test patches in staging first.
15. **Use (man <cmd>), (<cmd> --help), and (tldr <cmd>)** — the docs are on the box.

40. Practice Labs

Use a VM, a cloud free-tier instance, WSL, or a Docker container (`(docker run -it ubuntu bash)`).

1. **Navigation & files** — Create `(/opt/lab/{data, logs, conf})`, add files, copy/move/rename them, and create a symlink `(current → data)`.
2. **Permissions** — Create a user `(labuser)`. Make a script `(chmod 755)`, a secret file `(chmod 600)`, and confirm `(labuser)` can run the script but not read the secret. Fix ownership with `(chown)`.

3. **Text processing** — Generate a fake access log; use `grep`, `awk`, `sort | uniq -c` to find top IPs and count 404/500 responses.
 4. **Install & run a service** — Install `nginx`, enable it, check `systemctl status`, `ss -tulpn`, and `curl -I localhost`.
 5. **Firewall** — Enable UFW with SSH, HTTP, HTTPS only. Verify with `ufw status verbose`. Add then delete a custom port rule.
 6. **Custom systemd service** — Write a small app (Python `http.server` or Node) and create a unit file with `Restart=on-failure`. Kill the process and watch systemd restart it.
 7. **Monitoring** — Run a CPU stress test (`stress-ng` or `yes > /dev/null &`), observe with `htop`/`top`/`vmstat`, then stop it with `kill`.
 8. **SSH keys** — Generate an ed25519 key pair, install it on a second machine, disable password login, and test with `ssh -v`.
 9. **Cron/timers** — Schedule a job every minute that appends a timestamp to a file; then rebuild it as a systemd timer.
 10. **Disk** — Attach an extra disk/loop device, partition, format, mount, and add to `/etc/fstab` with a UUID. Test with `mount -a`.
 11. **Backup script** — Write a bash script that `tar`s a directory with a date stamp, keeps the last 7 backups, logs results, and runs from cron.
 12. **Troubleshooting drill** — Break something on purpose (stop service, wrong port, block with UFW, wrong file permission) and diagnose it using only the playbook in Section 25.
 13. **TLS** — Create a self-signed cert with `openssl`, configure Nginx for HTTPS, inspect it with `openssl s_client` and `curl -vk`.
 14. **Reverse proxy** — Run an app on port 3000, front it with Nginx, then break the app and observe the `502`; fix it using logs.
 15. **Docker** — Write a multi-stage Dockerfile for a small app, run it as non-root with a volume, and limit memory with `--memory`. Observe the UFW-bypass behavior of published ports.
 16. **Ansible** — Write a playbook that installs Nginx, enables it, and opens the firewall; run it with `--check` and then for real, twice, to confirm idempotency.
 17. **Backups** — Back up a PostgreSQL/MySQL database with a dump, delete the data, and restore it. Time how long recovery takes (your RTO).
 18. **strace** — Make an app fail due to a missing file or permission and find the exact cause with `strace -e trace=file`.
 19. **Recovery** — In a throwaway VM, break `/etc/fstab`, boot to rescue mode and repair it.
 20. **Kubernetes** — Deploy an app with a bad image tag, diagnose `ImagePullBackOff` with `kubectl describe`, fix it, and use `rollout undo`.
-

41. Interview Questions & Answers

- 1. What happens when you type `ls` and press Enter?** The shell parses the line, searches `$PATH` for `ls`, forks a child process, executes the binary with `execve`, the program reads the directory via syscalls and writes to stdout, then exits with status 0 which the parent shell collects.
- 2. Difference between a process and a thread?** A process has its own address space; threads within a process share memory and file descriptors but have separate stacks and registers.
- 3. Hard link vs symbolic link?** A hard link is another directory entry for the same inode (same filesystem only; survives deletion of the original name). A symlink is a separate file containing a path; it breaks if the target is removed and can cross filesystems.
- 4. What does `chmod 755` mean?** Owner `rwX`, group `r-X`, others `r-X`.
- 5. `kill` vs `kill -9`?** `kill` sends SIGTERM so the process can clean up; `kill -9` sends SIGKILL, which can't be caught or ignored and gives no cleanup chance.
- 6. Server load average is 8 on a 4-core machine. What does it mean?** The run queue (plus uninterruptible IO waiters) is about twice the core count — the system is overloaded. Check `top` for CPU hogs vs high `wa` (IO wait), and `vmstat` for swapping.
- 7. `df -h` shows free space but you can't create files. Why?** Possible inode exhaustion (`df -i`), a quota, a read-only remount after errors (`mount | grep ro`), or deleted files still held open.
- 8. A service is running but you can't reach it from outside. Steps?** `ss -tulpn` (bound to 127.0.0.1 or 0.0.0.0?), `curl localhost:port`, host firewall (`ufw status`), cloud security group/NACL, routing/DNS, and for Docker, the published ports.
- 9. How do you find what is using a port?** `sudo ss -tulpn | grep :PORT` or `sudo lsof -i :PORT`.
- 10. Explain the Linux boot process.** Firmware → bootloader (GRUB) → kernel + initramfs → systemd (PID 1) → default target → services → login prompt.
- 11. What is the OOM killer?** When memory is exhausted, the kernel kills a process (chosen by `oom_score`) to recover. Evidence: `dmesg -T | grep -i oom`. Fix by adding RAM, tuning limits, or fixing leaks.
- 12. `reload` vs `restart` in systemd?** `reload` asks the service to re-read config without stopping; `restart` stops then starts it (brief downtime, new PID).
- 13. How do containers use Linux features?** Namespaces isolate views (PID, net, mount, UTS, IPC, user); cgroups limit resources; overlay filesystems provide image layers; capabilities and seccomp restrict privileges.

14. How do you secure SSH? Key-only auth, `PermitRootLogin no`, `PasswordAuthentication no`, restrict users and source IPs, change/limit exposure (bastion/VPN), `fail2ban`, keep OpenSSH updated.

15. How do you safely run a destructive command on production? Dry-run first, back up, verify the target path/variables, run in a second session with rollback ready, change during a window, and log what you did.

16. What is idempotency and why does it matter? Running an operation repeatedly yields the same end state. It makes automation (Ansible, Terraform, scripts) safe to re-run and recover from partial failures.

17. How would you debug a process that hangs? `ps`/`top` state (D = IO wait, S = sleeping), `strace -p`, `lsof -p`, `/proc/<PID>/stack`, check dependent services (DB, network, DNS), and logs.

18. Why shouldn't you use `chmod 777`? It lets any user modify or replace the file — a security and integrity risk. Fix ownership/group instead.

42. Glossary

Term	Meaning
Kernel	Core of the OS: manages CPU, memory, devices, syscalls
Shell	Command interpreter ((<code>bash</code>), (<code>zsh</code>), (<code>sh</code>))
Daemon	Background service process (e.g. (<code>sshd</code>), (<code>nginx</code>))
PID / PPID	Process ID / parent process ID
UID / GID	User ID / Group ID (root = UID 0)
Inode	Filesystem structure storing file metadata (not the name)
Mount point	Directory where a filesystem is attached
Package repository	Server hosting installable packages
systemd unit	Config object managed by systemd (service, timer, socket, mount)
Signal	Asynchronous message to a process (SIGTERM, SIGKILL, SIGHUP)
stdin/stdout/stderr	Standard input, output, and error streams (fd 0, 1, 2)
Environment variable	Named value inherited by child processes
Bastion host	Hardened jump server for SSH access to private networks
Reverse proxy	Server that forwards client requests to backend servers
TLS	Transport Layer Security — encryption for network traffic
cgroup	Kernel feature limiting/accounting resource usage
Namespace	Kernel feature isolating system resources per process group
SELinux / AppArmor	Mandatory access control frameworks
IaC	Infrastructure as Code
Idempotent	Safe to run repeatedly with the same result
RPO / RTO	Recovery Point / Recovery Time Objective
MTTR	Mean Time To Recovery
SLI / SLO / SLA	Service level indicator / objective / agreement

Term	Meaning
Immutable infrastructure	Replace servers instead of modifying them in place
CNI	Container Network Interface (Kubernetes networking plugins)

Suggested learning path

1. Master sections 3–8 (files, text tools, permissions).
2. Services, logs & monitoring (10–12, 22).
3. Networking, firewall & SSH (14–16).
4. Scripting & automation (19–21).
5. Security hardening & troubleshooting (23, 25).
6. Web, TLS & tooling (26–30): time/NTP, certificates, Nginx, Git, Docker.
7. Platform & reliability (31–37): kubectl, backups, advanced debugging, SELinux/AppArmor, boot/recovery, IaC, CI/CD.
8. Container/Kubernetes internals (24) → then go deeper into Kubernetes, Terraform and observability.

Remember: Linux fluency is the foundation of DevOps. Every container, pod, pipeline, and cloud VM eventually comes down to processes, files, permissions, and networks — the topics covered here.